# I have a workflow that aims to help me merge CSV data from two files into one. When doing so I'll identify records by a key element, generally 'objectid', and need some strategies for presenting matching data versus new data, versus differences between new and old values.

A good way to handle this is to treat the merge as three separate outcomes: records that match on `objectid`, records that exist only in the new file, and matching records whose field values changed. Tools that compare CSVs or merge by a unique key commonly rely on a stable join key and then either preserve, add, or overwrite fields depending on the merge strategy.[1] [2]

## Suggested result categories

Use a status field such as:

- `match` for rows where `objectid` exists in both files and the compared values are identical.
- `new` for rows where `objectid` appears only in the incoming file.
- `changed` for rows where `objectid` exists in both files but one or more compared fields differ.
- `missing_in_new` if you also want to see rows that were in the old file but not in the new one.

That gives you a clean way to filter and review records after the merge, instead of trying to force everything into one combined view.[3] [4]

## Useful output columns

A practical merged output often includes:

- `objectid`
- one or more key identity fields
- old value columns, prefixed like `old_name`, `old_date`
- new value columns, prefixed like `new_name`, `new_date`
- a `status` column
- optional `changed_fields` listing which columns differ

For example, if `title` changed from `Old Bridge` to `Bridge Renovated`, you can show both values side by side and mark the row `changed`. That is usually easier to audit than overwriting values immediately.[5] [4]

## Presentation strategies

If you want the output to be easy to read in Excel or a script, these patterns work well:

- Side-by-side comparison: keep both old and new columns.
- Diff-only view: show only rows with differences.
- Summary view: count how many are `match`, `new`, and `changed`.
- Change mask: add a boolean column per field, like `title_changed = yes/no`.

For human review, side-by-side is best. For automation, a `status` column plus per-field change flags is usually enough.[6] [4]

## Merge logic

A simple merge rule is:

- Match on `objectid`.
- If the key exists in both files, compare selected fields.
- If all compared fields are equal, mark `match`.
- If any differ, mark `changed` and preserve both versions.
- If the key exists only in the new file, mark `new`.

This aligns with standard join-based CSV workflows, where the key is used to align rows and decide whether a record is added, preserved, or updated.[7] [8]

## Recommended workflow

1. Normalize `objectid` first so both files use the same type and formatting.
2. Decide which fields matter for change detection.
3. Merge on `objectid`.
4. Add a status column.
5. Add old/new columns for any fields you want to inspect.
6. Export a review file plus, optionally, a final cleaned file.

That approach keeps your comparison transparent and makes it easy to separate true updates from brand-new records.[2] [6]

### Example layout

| objectid | status | old_title | new_title | title_changed |
|---|---|---|---|---|
| 101 | match | Trail Cache | Trail Cache | no |
| 102 | changed | Old Bridge | Bridge Renovated | yes |
| 103 | new | | New Sign | yes |

That structure is compact, easy to sort, and makes the differences obvious at a glance.[4]

Would you like a Python/pandas template that produces this kind of CSV directly?

⁂

# Yes, I would like a Python/Pandas template to help implement all of this, and a prompt I can feed to my VSCode/GitHub Copilot to help flesh it out on another platform.

Yes — below is a reusable pandas template plus a Copilot prompt you can drop into VSCode. In pandas, the key pieces for this workflow are `merge(..., indicator=True, suffixes=...)` to classify row origin and `validate='one_to_one'` to enforce key uniqueness when that assumption should hold.[16] [17]

### Python template

This template reads an "old" CSV and a "new" CSV, merges on `objectid`, labels rows as `match`, `changed`, `new`, or `missing_in_new`, and produces both a full review file and a changes-only file. `indicator=True` adds merge-source tracking, while suffixes preserve old and new versions of overlapping columns for side-by-side comparison.[18] [19] [16]

```
import pandas as pd
from pathlib import Path

# ----------------------------
# Configuration
# ----------------------------
OLD_FILE = "old.csv"
NEW_FILE = "new.csv"
KEY = "objectid"

# Set to None to auto-compare all shared non-key columns
COMPARE_COLUMNS = None

# Optional: columns to ignore even if they exist in both files
IGNORE_COLUMNS = {
    # "last_modified",
    # "timestamp",
}
```

```
# Optional normalization rules
STRIP_WHITESPACE = True
CASE_INSENSITIVE_COMPARE = False

# Output files
OUTPUT_ALL = "merged_review.csv"
OUTPUT_CHANGES = "merged_changes_only.csv"
OUTPUT_SUMMARY = "merge_summary.csv"


# ----------------------------
# Helpers
# ----------------------------
def normalize_key(series: pd.Series) -> pd.Series:
    s = series.astype("string")
    if STRIP_WHITESPACE:
        s = s.str.strip()
    return s

def normalize_compare(series: pd.Series) -> pd.Series:
    s = series.astype("string")
    if STRIP_WHITESPACE:
        s = s.str.strip()
    if CASE_INSENSITIVE_COMPARE:
        s = s.str.lower()
    return s.fillna("")

def ensure_unique_key(df: pd.DataFrame, key: str, label: str):
    dupes = df[df.duplicated(subset=[key], keep=False)].sort_values(by=[key])
    if not dupes.empty:
        raise ValueError(
            f"{label} contains duplicate key values in '{key}'. "
            f"Fix duplicates before merging.\n{dupes[[key]].drop_duplicates().head(2
        )

def choose_compare_columns(old_df: pd.DataFrame, new_df: pd.DataFrame, key: str):
    shared = set(old_df.columns).intersection(new_df.columns)
    shared.discard(key)
    shared -= set(IGNORE_COLUMNS)
    return sorted(shared)


# ----------------------------
# Load data
# ----------------------------
old_df = pd.read_csv(OLD_FILE, dtype="string")
new_df = pd.read_csv(NEW_FILE, dtype="string")

if KEY not in old_df.columns or KEY not in new_df.columns:
    raise KeyError(f"Key column '{KEY}' must exist in both files.")

old_df[KEY] = normalize_key(old_df[KEY])
new_df[KEY] = normalize_key(new_df[KEY])

ensure_unique_key(old_df, KEY, "Old file")
```

```
ensure_unique_key(new_df, KEY, "New file")

compare_cols = COMPARE_COLUMNS or choose_compare_columns(old_df, new_df, KEY)

# Normalize comparison columns without overwriting originals
old_norm = old_df.copy()
new_norm = new_df.copy()
for col in compare_cols:
    old_norm[col] = normalize_compare(old_norm[col]) if col in old_norm.columns els
    new_norm[col] = normalize_compare(new_norm[col]) if col in new_norm.columns els

# ----------------------------
# Merge
# ----------------------------
merged = old_df.merge(
    new_df,
    on=KEY,
    how="outer",
    suffixes=("_old", "_new"),
    indicator=True,
    validate="one_to_one",
)

merged_norm = old_norm.merge(
    new_norm,
    on=KEY,
    how="outer",
    suffixes=("_old", "_new"),
    indicator=False,
    validate="one_to_one",
)

# ----------------------------
# Row status classification
# ----------------------------
def classify_row(row):
    if row["_merge"] == "left_only":
        return "missing_in_new"
    if row["_merge"] == "right_only":
        return "new"

    changed_fields = []
    for col in compare_cols:
        old_val = row.get(f"{col}_old", "")
        new_val = row.get(f"{col}_new", "")
        if pd.isna(old_val):
            old_val = ""
        if pd.isna(new_val):
            new_val = ""
        if old_val != new_val:
            changed_fields.append(col)

    return "changed" if changed_fields else "match"

statuses = []
changed_fields_list = []
```

```
for _, row in merged_norm.iterrows():
    if row["_merge"] == "left_only":
        statuses.append("missing_in_new")
        changed_fields_list.append("")
    elif row["_merge"] == "right_only":
        statuses.append("new")
        changed_fields_list.append(",".join(compare_cols))
    else:
        changed = []
        for col in compare_cols:
            old_val = row.get(f"{col}_old", "")
            new_val = row.get(f"{col}_new", "")
            if pd.isna(old_val):
                old_val = ""
            if pd.isna(new_val):
                new_val = ""
            if old_val != new_val:
                changed.append(col)
        statuses.append("changed" if changed else "match")
        changed_fields_list.append(",".join(changed))

merged["status"] = statuses
merged["changed_fields"] = changed_fields_list

# ----------------------------
# Optional per-column flags
# ----------------------------
for col in compare_cols:
    old_col = f"{col}_old"
    new_col = f"{col}_new"
    flag_col = f"{col}_changed"

    old_vals = merged_norm.get(old_col, pd.Series("", index=merged.index)).fillna(
    new_vals = merged_norm.get(new_col, pd.Series("", index=merged.index)).fillna(
    merged[flag_col] = old_vals != new_vals

# ----------------------------
# Reorder columns
# ----------------------------
base_cols = [KEY, "status", "_merge", "changed_fields"]
other_cols = [c for c in merged.columns if c not in base_cols]
merged = merged[base_cols + other_cols]

# ----------------------------
# Write outputs
# ----------------------------
merged.to_csv(OUTPUT_ALL, index=False)

changes_only = merged[merged["status"].isin(["changed", "new", "missing_in_new"])].
changes_only.to_csv(OUTPUT_CHANGES, index=False)

summary = (
    merged["status"]
    .value_counts(dropna=False)
    .rename_axis("status")
```

```
    .reset_index(name="count")
)
summary.to_csv(OUTPUT_SUMMARY, index=False)

print(f"Wrote {OUTPUT_ALL}, {OUTPUT_CHANGES}, and {OUTPUT_SUMMARY}")
```

## What it does

`merge(... how='outer' ...)` keeps all rows from both files so you can distinguish rows found only in the old file, only in the new file, or in both files, and the merge indicator identifies those origins directly. `DataFrame.compare()` also exists for side-by-side difference inspection, but for a CSV review workflow a merged table with `_old` and `_new` columns is usually easier to filter, sort, and export.[20] [21] [16]

A good pattern is to review these outputs:

- `merged_review.csv` — everything, including matches.
- `merged_changes_only.csv` — just `new`, `changed`, and `missing_in_new`.
- `merge_summary.csv` — counts by status.

## Practical adjustments

If your CSVs sometimes contain duplicate `objectid` values, keep the duplicate check and fail early, because `validate='one_to_one'` is specifically meant to verify that the merge keys are unique on both sides. If you want values like `"ABC "` and `"abc"` treated as equal, normalize with whitespace stripping and optional lowercase conversion before comparison, but preserve the original columns in the final export so reviewers still see the real incoming values.[17] [22] [16]

You can also tune behavior by:

- Setting `COMPARE_COLUMNS` explicitly when only some fields matter.
- Adding audit columns like `source_file`, `run_date`, or `review_notes`.
- Changing `missing_in_new` to a friendlier label like `retired` or `not_in_feed`.

## Copilot prompt

Here is a prompt you can paste into VSCode or GitHub Copilot:

```
Help me build a Python 3 script using pandas to merge two CSV files by a unique key

Requirements:
- Read two CSV files: an "old" file and a "new" file.
- Merge with an outer join on objectid.
- Preserve overlapping columns using suffixes _old and _new.
- Add a merge indicator column so I can tell whether a row came from both files, onl
- Validate that objectid is unique in both files and fail with a clear error if dupl
- Compare selected columns, or auto-detect all shared non-key columns if no list is
- Classify each row with a status:
```

```
  - match = objectid exists in both files and all compared fields are equal
  - changed = objectid exists in both files and one or more compared fields differ
  - new = objectid exists only in the new file
  - missing_in_new = objectid exists only in the old file
- Add a changed_fields column listing the names of fields that changed.
- Optionally add boolean per-column flags like title_changed, date_changed, etc.
- Normalize comparison values by trimming whitespace, and optionally compare case-i
- Write three output CSV files:
  1. full merged review file
  2. changes-only file
  3. status summary file with counts
- Structure the code into small functions with clear names.
- Use a main() function and argparse so I can run it from the command line.
- Make the script easy to adapt for other key names and ignored columns.
- Include comments explaining the merge logic and status classification.
- Add an option to keep only selected output columns in the final review CSV.
- Use clean, readable pandas code rather than one-liners.

Please generate the full script, including imports, helper functions, argparse, and
```

## Command-line shape

A nice next step is to wrap the script with `argparse` so you can run something like this:

```
python merge_csvs.py \
  --old old.csv \
  --new new.csv \
  --key objectid \
  --compare title creator date \
  --ignore last_modified timestamp \
  --output-dir output
```

That kind of interface makes the workflow reusable across projects and keeps the comparison rules explicit in each run.[16] [17]

Would you like me to turn this into a finished `merge_csvs.py` file with `argparse` and sample test data?

⁂


1. https://thetexttool.com/blog/csv-merge-complete-guide
2. https://learn.microsoft.com/en-us/power-query/combine-files-csv
3. https://help.ortto.com/a-299-data-source-merge-strategies
4. https://www.altova.com/blog/2022/05/how-to-compare-csv-files-or-compare-a-csv-file-to-a-database-table
5. https://stackoverflow.com/questions/69108584/how-to-compare-two-columns-in-two-different-csvs-old-and-new-and-update-a-thi
6. https://csv-loader.com/csv-guide/how-to-compare-two-csv-files
7. https://stackoverflow.com/questions/45068584/merge-two-csv-files-into-one-with-pandas-by-id

8. https://www.datablist.com/how-to/join-csv-files-unique-identifier-or-column
9. https://www.csv-merge.com/merge-csv-by-id/
10. https://www.tutorialspoint.com/article/how-to-merge-two-csv-files-by-specific-column-using-pandas-in-python
11. https://community.neo4j.com/t/matching-and-merging-data-from-multiple-csv-files/58666
12. https://www.reddit.com/r/PowerShell/comments/tew0r7/compare_and_merge_differences_in_2_csvs/
13. https://www.reddit.com/r/excel/comments/12r6018/i_want_to_merge_multiple_csv_files_based_off_a/
14. https://help.formstack.com/hc/en-us/articles/44592104563731-Merge-Data-from-a-CSV-spreadsheet
15. https://www.reddit.com/r/learnprogramming/comments/9f1syh/how_can_i_merge_two_csv_files_with_a_common_id/
16. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.merge.html
17. https://pandas.pydata.org/docs/reference/api/pandas.merge.html
18. https://sparkbyexamples.com/pandas/pandas-merge-indicator-explained-with-examples/
19. https://sparkbyexamples.com/pandas/pandas-merge-suffixes-explained-examples/
20. https://www.reddit.com/r/learnpython/comments/154ewr8/is_this_the_most_efficient_way_to_compare_2/
21. https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.compare.html
22. https://campus.datacamp.com/courses/joining-data-with-pandas/advanced-merging-and-concatenating?ex=8
23. https://blogs.cornell.edu/jingyi/2020/04/03/python-pandas-merge-two-dataframes-based-on-multiple-columns/
24. https://scales.arabpsychology.com/stats/how-to-compare-columns-in-two-different-data-frames-in-pandas/
25. https://stackoverflow.com/questions/69275059/compare-values-between-dataframes-and-merge-columns-in-python
26. https://www.w3resource.com/python-exercises/pandas/python-pandas-data-frame-exercise-69.php
27. https://realpython.com/pandas-merge-join-and-concat/
28. https://www.geeksforgeeks.org/python/merge-two-pandas-dataframes-with-complex-conditions/
29. https://pandas.pydata.org/docs/user_guide/merging.html
30. https://stackoverflow.com/questions/51768693/how-to-compare-or-merge-two-data-frames-using-python-pandas